1. Multiplication Table Generator (Nested Loops)

Write a program that generates multiplication tables from 1 to 10.

- Use a nested loop: one loop for rows (1–10) and another for columns (1–10).
- Print each row on a new line.

In [2]:
```
# Solution 1:
for rows in range(1,11):
    for columns in range(1,11):
        print(f"{rows}*{columns}={rows*columns}", end="\t")
    print()
```

```
1*1=1   1*2=2   1*3=3   1*4=4   1*5=5   1*6=6   1*7=7   1*8=8   1*9=9   1*10=10
2*1=2   2*2=4   2*3=6   2*4=8   2*5=10  2*6=12  2*7=14  2*8=16  2*9=18  2*10=20
3*1=3   3*2=6   3*3=9   3*4=12  3*5=15  3*6=18  3*7=21  3*8=24  3*9=27  3*10=30
4*1=4   4*2=8   4*3=12  4*4=16  4*5=20  4*6=24  4*7=28  4*8=32  4*9=36  4*10=40
5*1=5   5*2=10  5*3=15  5*4=20  5*5=25  5*6=30  5*7=35  5*8=40  5*9=45  5*10=50
6*1=6   6*2=12  6*3=18  6*4=24  6*5=30  6*6=36  6*7=42  6*8=48  6*9=54  6*10=60
7*1=7   7*2=14  7*3=21  7*4=28  7*5=35  7*6=42  7*7=49  7*8=56  7*9=63  7*10=70
8*1=8   8*2=16  8*3=24  8*4=32  8*5=40  8*6=48  8*7=56  8*8=64  8*9=72  8*10=80
9*1=9   9*2=18  9*3=27  9*4=36  9*5=45  9*6=54  9*7=63  9*8=72  9*9=81  9*10=90
10*1=10 10*2=20 10*3=30 10*4=40 10*5=50 10*6=60 10*7=70 10*8=80 10*9=90 10*10=100
```

2. Prime Numbers Finder

Write a program to display all prime numbers between 1 and 100.

- Use a loop to iterate through numbers.
- Use an inner loop or if statements to check if a number is divisible only by 1 and itself.

In [8]:
```
#Solution 2:
number=0
counter=0
print("Prime numbers between 1 to 100 are: ")
while number<=100:
    for count in range (1,number+1):
        if number%count==0:
            counter+=1
    if counter==2:
```

```
        print(number, end=" ")

    number+=1
    counter=0
```

```
Prime numbers between 1 to 100 are:
2 3 5 7 11 13 17 19 23 29 31 37 41 43 47 53 59 61 67 71 73 79 83 89 97
```

3. Seating Arrangement Checker (Nested Loops)

Simulate a classroom seating arrangement.

- There are 3 rows, each with 4 seats.
- Display a seating chart like:
    - Row 1: Seat 1, Seat 2, Seat 3, Seat 4
    - Row 2: Seat 1, Seat 2, Seat 3, Seat 4

In [3]:
```
# Solution 3
for row in range(1,4):
    print("Row", row, ":", end=" ")
    for seat in range(1,5):
        if seat < 4:
            print(f"Seat {seat}", end=", ")
        else:
            print(f"Seat {seat}", end=" ")
    print()
```

```
Row 1 : Seat 1, Seat 2, Seat 3, Seat 4
Row 2 : Seat 1, Seat 2, Seat 3, Seat 4
Row 3 : Seat 1, Seat 2, Seat 3, Seat 4
```

4. Unique Word Counter

Write a program to count unique words in a given sentence.

- Use a loop to split the input string into words.
- Use an if condition to check if a word already exists in a list.

In [38]:
```
# Solution 4
words=[]
word=""
```

```
sentence=input("Enter the sentence").lower()
sentence=sentence+" "
for character in sentence:
    if character.isalpha(): #to find letters only
        word += character
    else:
        if word !="":
            if word not in words: #if condition to check the existence of any word in list
                words.append(word)
        word=""
print("Number of unique words are:",len(words))
```

```
Number of unique words are: 5
```

5. Library Book Management

A library system has 5 racks with up to 10 books each.

- Use nested loops to list the books in each rack.
- Ask the user to search for a book and display which rack it's in.

In [41]:
```
# Solution 5
library = [
    ["Harry Potter", "Atomic Habits", "Ikigai", "The Alchemist", "Rich Dad Poor Dad",
     "Think and Grow Rich", "1984", "Animal Farm", "The Hobbit", "Dune"],

    ["To Kill a Mockingbird", "Pride and Prejudice", "Jane Eyre", "Wings of Fire", "The Kite Runner",
     "The Great Gatsby", "The Catcher in the Rye", "Brave New World", "The Da Vinci Code", "Inferno"],

    ["The Silent Patient", "Verity", "Gone Girl", "The Girl on the Train", "It Ends With Us",
     "It Starts With Us", "The Fault in Our Stars", "Looking for Alaska", "Twisted Love", "Twisted Games"],

    ["Python Basics", "Automate the Boring Stuff", "Clean Code", "Introduction to Algorithms", "Cracking the Coding Interview",
     "Data Structures in Python", "Head First Python", "Effective Python", "Python Crash Course", "Fluent Python"],

    ["Sapiens", "Homo Deus", "Deep Work", "The Psychology of Money", "Zero to One",
     "The Lean Startup", "Steve Jobs", "Educated", "Becoming", "Can't Hurt Me"]
]

for rack in range(5): # displaying books list
    print(f"Rack {rack+1}:", end="")
    for book in library[rack]:
        print(book,end=", ")
    print()

enter_book = input("Please find me the book: ").lower()
found=False
```

```python
for i in range(5):
    for j in library[i]:
        if j.lower()==enter_book:
            print(f"Found in Rack{i+1}")
            found=True
if not found:
    print("Not found")
```

```
Rack 1:Harry Potter, Atomic Habits, Ikigai, The Alchemist, Rich Dad Poor Dad, Think and Grow Rich, 1984, Animal Farm, The Hobbit, Dune,
Rack 2:To Kill a Mockingbird, Pride and Prejudice, Jane Eyre, Wings of Fire, The Kite Runner, The Great Gatsby, The Catcher in the Rye, Brave New World, The Da Vi
nci Code, Inferno,
Rack 3:The Silent Patient, Verity, Gone Girl, The Girl on the Train, It Ends With Us, It Starts With Us, The Fault in Our Stars, Looking for Alaska, Twisted Love,
Twisted Games,
Rack 4:Python Basics, Automate the Boring Stuff, Clean Code, Introduction to Algorithms, Cracking the Coding Interview, Data Structures in Python, Head First Pyth
on, Effective Python, Python Crash Course, Fluent Python,
Rack 5:Sapiens, Homo Deus, Deep Work, The Psychology of Money, Zero to One, The Lean Startup, Steve Jobs, Educated, Becoming, Can't Hurt Me,
Found in Rack5
```

6. Pattern Printing (Triangle)

Write a program to print the following triangle pattern using nested loops:

```
1
1 2
1 2 3
1 2 3 4
1 2 3 4 5
```

In [15]:
```python
# Solution 6
for i in range(1,6):
    for j in range(1,i+1):
        print(j,end=" ")
    print("")
```

```
1
1 2
1 2 3
1 2 3 4
1 2 3 4 5
```

7. Temperature Comparison Across Cities

Simulate a weather app that compares temperatures across 3 cities for 7 days.

- Use nested loops to iterate over cities and days.
- Display the highest and lowest temperatures for each city.

In [45]:
```
#solution 7
cities = ["Chennai", "Delhi", "Kolkata"]

temperatures = [[32, 33, 31, 30, 34, 35, 33],[38, 40, 39, 41, 42, 40, 39],[34, 35, 33, 36, 37, 35, 34]]

for i in range(3):
    highest = temperatures[i][0]
    lowest = temperatures[i][0]

    for temp in temperatures[i]:
        if temp > highest:
            highest = temp

        if temp < lowest:
            lowest = temp

    print(f"{cities[i]}")
    print(f"Highest Temperature: {highest} °C")
    print(f"Lowest Temperature: {lowest} °C")
    print()
```

```
Chennai
Highest Temperature: 35 °C
Lowest Temperature: 30 °C

Delhi
Highest Temperature: 42 °C
Lowest Temperature: 38 °C

Kolkata
Highest Temperature: 37 °C
Lowest Temperature: 33 °C
```

8. Password Strength Checker

Write a program to check the strength of multiple passwords.

- Use a loop to validate a list of passwords.
- Use conditions to check: length, special characters, and numbers.

In [58]:
```python
# Solution 8
passwords=["pritamdas@123","ABCabc@#$123","123456789","abc123", "Strong@123", "weak", "Python#9","11111111"]
special="!@#$%^&*()_+-"
for password in passwords:
    isdigit,isalpha,isspecial=0,0,0
    if len(password)>=8:
        for character in  password:
            if character.isdigit():
                isdigit=1
            if character.isalpha():
                isalpha=1
            if character in special:
                isspecial=1
        if isdigit==1 and isalpha==1 and isspecial==1:
            print(f"{password} is a Strong password")
        else:
            print(f"{password} is a Medium password")
    else:
        print(f"{password} is a Weak password")
```

```
pritamdas@123 is a Strong password
ABCabc@#$123 is a Strong password
123456789 is a Medium password
abc123 is a Weak password
Strong@123 is a Strong password
weak is a Weak password
Python#9 is a Strong password
11111111 is a Medium password
```

9. Inventory Tracker (Nested Loops)

Create a program to manage stock levels in a warehouse with multiple sections.

- Each section has 5 items.
- Display items running low (<5 units) and their section numbers.

In [46]:
```python
#slution 9
total_stock = [[10, 3, 7, 2, 8],[4, 9, 6, 1, 12],[15, 2, 5, 3, 7]]
print("Items running low are: ")
for section in range(len(total_stock)):
    for item in range(len(total_stock[section])):
        if total_stock[section][item] <5:
            print(f"Section: {section+1}, Item number: {item+1}, Units remaining: {total_stock[section][item]}")
```

```
Items running low are:
Section: 1, Item number: 2, Units remaining: 3
Section: 1, Item number: 4, Units remaining: 2
Section: 2, Item number: 1, Units remaining: 4
Section: 2, Item number: 4, Units remaining: 1
Section: 3, Item number: 2, Units remaining: 2
Section: 3, Item number: 4, Units remaining: 3
```

10. Tic-Tac-Toe Grid

Simulate a 3x3 Tic-Tac-Toe grid.

- Use nested loops to create and display the grid.
- Allow the user to mark cells (e.g., "X" or "O") by inputting row and column numbers.

In [ ]:
```python
#solution 10
board = [
    [" ", " ", " "],
    [" ", " ", " "],
    [" ", " ", " "],
]

current_player = "X"
turns = 0
game_over = False
```

```
print("Welcome to Tic-Tac-Toe!")

# --- STEP 2: Main Game Loop ---
# Keep playing as long as the game isn't over and we haven't hit 9 turns
while turns < 9 and not game_over:


    print("\n  0   1   2")  # Column numbers header
    print(" ---+---+---")
    for i in range(3):
        # 'i' serves as our row number (0, 1, or 2) on the left side
        print(f"{i} {board[i][0]} | {board[i][1]} | {board[i][2]}")
        print(" ---+---+---")

    print(f"Player {current_player}'s turn.")

    # Get player's choice
    try:
        row = int(input("Enter row number (0, 1, or 2): "))
        col = int(input("Enter column number (0, 1, or 2): "))
    except ValueError:
        print("Oops! Please enter numbers only.")
        continue

    # Check if the move is allowed
    if row < 0 or row > 2 or col < 0 or col > 2:
        print("Oops! That's outside the board. Pick 0, 1, or 2.")
        continue  # Restart the loop to let them try again

    if board[row][col] != " ":
        print("That spot is already taken! Try again.")
        continue  # Restart the loop to let them try again

    # Place the mark
    board[row][col] = current_player
    turns = turns + 1  # Count this turn

    # Check for a winner (Simple If Statements)

    # Check Rows (Horizontal)
    for i in range(3):
        if (
            board[i][0] == current_player
            and board[i][1] == current_player
            and board[i][2] == current_player
        ):
            game_over = True

    # Check Columns (Vertical)
    for i in range(3):
        if (
```

```
            board[0][i] == current_player
            and board[1][i] == current_player
            and board[2][i] == current_player
        ):
            game_over = True

    # Check Diagonals (Slanted lines)
    if (
        board[0][0] == current_player
        and board[1][1] == current_player
        and board[2][2] == current_player
    ):
        game_over = True
    if (
        board[0][2] == current_player
        and board[1][1] == current_player
        and board[2][0] == current_player
    ):
        game_over = True

    # Wrap up the turn
    if game_over:
        # Print the final board one last time with row numbers
        print("\n  0   1   2")
        print(" ---+---+---")
        for i in range(3):
            print(f"{i} {board[i][0]} | {board[i][1]} | {board[i][2]}")
            print(" ---+---+---")
        print(f"\nPlayer {current_player} wins the game! 🎉")
    else:
        # Swap players: if X just went, it's O's turn. Otherwise, it's X's turn.
        if current_player == "X":
            current_player = "O"
        else:
            current_player = "X"

# Checking for a Tie or draw

if turns == 9 and not game_over:
    print("\n  0   1   2")
    print(" ---+---+---")
    for i in range(3):
        print(f"{i} {board[i][0]} | {board[i][1]} | {board[i][2]}")
        print(" ---+---+---")
    print("\nIt's a tie! No one wins. 🤝")
```

11. Palindrome Checker

Write a program to check if multiple strings from a list are palindromes.

- Use a loop to iterate through the list.
- Use if to compare each string with its reverse.

In [61]:
```python
#Solution 11
string_list = ["madam", "hello", "racecar", "python", "level","Madam"]
for word in string_list:
    w=word.lower()
    if w== w[::-1]:
        print(f"{word} is a Palindrome")
    else:
        print(f"{word} is not a palindrome")
```

```
madam is a Palindrome
hello is not a palindrome
racecar is a Palindrome
python is not a palindrome
level is a Palindrome
Madam is a Palindrome
```

12. Employee Attendance Tracker

Simulate tracking attendance for 10 employees across 5 days.

- Use nested loops to record attendance (P for present, A for absent).
- Display a summary of each employee's attendance at the end.

In [52]:
```python
# solution 12
employees = ["Aarav Kumar","Bob Biswas","Ishaan Gupta","Pritam Das","Rohan Singh","George Hanks","Gaurav GUpta","Tom Hanks","Ivy Sen","Meera Thakur"]

attendance_summary = {}

print("--- Employee Attendance Tracker ---")
print("Enter 'P' for Present and 'A' for Absent.\n")

for name in employees:
    print(f"Recording attendance for Employee No.{employees.index(name)+1}: {name}:")
    present_count = 0  # Reset the count for the new employee
```

```
    day = 1
    while day <= 5:
        status = input(f"  Day {day}: ").lower()


        if status != "p" and status != "a":# Checking valid letters
            print("  Invalid input! Please enter only 'P' or 'A'.")
            continue  # Restart the while loop for the same day


        if status == "p": #counting present days
            present_count = present_count + 1

        day = day + 1

    # Store the final count in our summary tracker
    attendance_summary[name] = present_count
    print("-" * 30)

print("FINAL ATTENDANCE SUMMARY: ")

for name in employees:
    days_present = attendance_summary[name]
    days_absent = 5 - days_present
    print(f"{name}: Present = {days_present} days | Absent = {days_absent} days")
```

```
--- Employee Attendance Tracker ---
Enter 'P' for Present and 'A' for Absent.

Recording attendance for Employee No.1: Aarav Kumar:
------------------------------
Recording attendance for Employee No.2: Bob Biswas:
------------------------------
Recording attendance for Employee No.3: Ishaan Gupta:
  Invalid input! Please enter only 'P' or 'A'.
------------------------------
Recording attendance for Employee No.4: Pritam Das:
------------------------------
Recording attendance for Employee No.5: Rohan Singh:
------------------------------
Recording attendance for Employee No.6: George Hanks:
------------------------------
Recording attendance for Employee No.7: Gaurav GUpta:
------------------------------
Recording attendance for Employee No.8: Tom Hanks:
------------------------------
Recording attendance for Employee No.9: Ivy Sen:
------------------------------
Recording attendance for Employee No.10: Meera Thakur:
```

```
----------------------------
FINAL ATTENDANCE SUMMARY:
Aarav Kumar: Present = 5 days | Absent = 0 days
Bob Biswas: Present = 0 days | Absent = 5 days
Ishaan Gupta: Present = 5 days | Absent = 0 days
Pritam Das: Present = 3 days | Absent = 2 days
Rohan Singh: Present = 3 days | Absent = 2 days
George Hanks: Present = 0 days | Absent = 5 days
Gaurav GUpta: Present = 0 days | Absent = 5 days
Tom Hanks: Present = 4 days | Absent = 1 days
Ivy Sen: Present = 2 days | Absent = 3 days
Meera Thakur: Present = 5 days | Absent = 0 days
```

13. Grade Categorizer

Write a program to categorize student grades.

- Input grades for 5 students using a loop.
- Use if conditions to assign letter grades (A, B, C, etc.) and display the results.

In [70]:
```python
#Solution 13
marks=[]
for i in range(0,5):
    mark=int(input(f"Please enter the marks of student {i+1}: "))
    marks.append(mark)

for grade in marks:
    if grade >= 90:
        verdict = "A"
    elif grade >= 80:
        verdict = "B"
    elif grade >= 70:
        verdict = "C"
    elif grade >= 60:
        verdict = "D"
    else:
        verdict = "F"
    print(f"Grade for {grade} is:", verdict)
```

```
Grade for 98 is: A
Grade for 77 is: C
Grade for 89 is: B
Grade for 88 is: B
Grade for 96 is: A
```

14. Pyramid Pattern (Numbers)

Write a program to print the following pattern:

1
1 2
1 2 3
1 2 3 4

In [2]:
```
#solution 14
n=4
for i in range(1,n+1):
    print(" "*(n-i),end="")
    for j in range(1, i+1):
        print(j,end=" ")

    print()
```

```
   1
  1 2
 1 2 3
1 2 3 4
```

15. Sales Data Analysis

Analyze sales data for 3 products over 4 months.

- Use nested loops to store and display sales data.
- Highlight the product with the highest sales each month.

In [58]:
```
#solution 15
products = ["Laptop", "Smartphone", "Headphones"]
months = ["January", "February", "March", "April"]
sales_data = {}

for month in months:
    print(f"\nEntering data for {month}:")

    sales_data[month] = {}
```

```
    for product in products:
        try:
            sales = int(input(f"  Enter sales units for {product}: "))
        except ValueError:
            print("  Invalid input! Setting sales to 0 for this item.")
            sales = 0

        # Store the sales number under the specific product for this month
        sales_data[month][product] = sales


print("\nMONTHLY SALES REPORT:-")

# Loop through each month to read the data back out
for month in months:
    # Track the best product of the month
    highest_sales = -1
    best_product = ""

    for product in products:
        current_sales = sales_data[month][product]

        if current_sales > highest_sales:
            highest_sales = current_sales
            best_product = product

    # Highlight the winner for this month
    print(f"Best Seller for the month {month.upper()}: {best_product} ({highest_sales} units)")
#NOTE: if two or more products have same unit sales, then the first one in the list shall be shown
```

```
Entering data for January:
Entering data for February:
Entering data for March:
Entering data for April:
MONTHLY SALES REPORT:-
Best Seller for the month JANUARY: Headphones (6 units)
Best Seller for the month FEBRUARY: Headphones (7 units)
Best Seller for the month MARCH: Laptop (6 units)
Best Seller for the month APRIL: Smartphone (9 units)
```

16. Palindrome Numbers in a Range

Write a program to find all palindrome numbers between 10 and 200.

- Use a loop to iterate through the range.
- Use if conditions to check if a number reads the same backward as forward.

In [17]:
```python
# Solution 16
for i in range(10,201):
    x=0
    original=i
    num=i
    while num>0:
        digit=num%10
        x=x*10+digit
        num=num//10
    if original==x:
        print(original, end=" ")
```

```
11 22 33 44 55 66 77 88 99 101 111 121 131 141 151 161 171 181 191
```

17. Magic Square Validator

Write a program to check if a given 3x3 matrix is a magic square (all rows, columns, and diagonals add up to the same number).

- Use nested loops to iterate through the matrix.
- Use if conditions to validate the sums.

In [61]:
```python
#solution 17
matrix = [
    [8, 1, 6],
    [3, 5, 7],
    [4, 9, 2],
]

magic_sum = matrix[0][0] + matrix[0][1] + matrix[0][2]
is_magic = True  # We assume it is magic until proven guilty!

for i in range(3):
    row_sum = 0
    for j in range(3):
        row_sum = row_sum + matrix[i][j]  # Add up elements in the row

    if row_sum != magic_sum:
        is_magic = False

for j in range(3):
    col_sum = 0
    for i in range(3):
        col_sum = col_sum + matrix[i][j]  # Add up elements in the column
```

```
    if col_sum != magic_sum:
        is_magic = False

diag1_sum = matrix[0][0] + matrix[1][1] + matrix[2][2]
if diag1_sum != magic_sum:
    is_magic = False

diag2_sum = matrix[0][2] + matrix[1][1] + matrix[2][0]
if diag2_sum != magic_sum:
    is_magic = False

if is_magic:
    print(f"It's a Magic Square! Every line adds up to {magic_sum}.")
else:
    print("Not a Magic Square. The sums do not match.")
```

```
It's a Magic Square! Every line adds up to 15.
```

18. Meal Planner

Simulate a meal planner for a week.

- Use a nested loop to list 7 days and 3 meals (breakfast, lunch, dinner).
- Allow the user to input or modify meals for each day.

In [64]:
```
# Solution 18
days = [
    "Monday",
    "Tuesday",
    "Wednesday",
    "Thursday",
    "Friday",
    "Saturday",
    "Sunday",
]
meals = ["Breakfast", "Lunch", "Dinner"]

specials = "!@#$%^&*()_+="
meal_plan = {}
print("Welcome to food planner simulator")
print("NOTE: Please do not input any special characters, or digits, as it will be considered as Food Not Planned")
for day in days:
    print(f"\nPlan for {day}:")

    meal_plan[day] = {}
```

```
    for meal in meals:

        food = input(f"What would you like for {meal}?: ")

        # FIXED: any(...) checks if *any* character inside the food string is one of the specials
        if (
            food == ""
            or food.isdigit()
            or any(char in specials for char in food)
        ):
            food = "Not planned"

        meal_plan[day][meal] = food


print("\nYOUR WEEKLY MEAL PLAN:-")

for day in days:
    print(f"\n📌 {day.upper()}")
    print("-" * 20)
    for meal in meals:
        print(f"  {meal}: {meal_plan[day][meal]}")


modify = input("\nWould you like to modify any meal? (yes/no): ").lower()

# Great addition here! This handles all variation inputs cleanly.
if modify == "yes" or modify == "y" or modify == "ya" or modify == "yeah":
    # Ask which day and meal to change
    target_day = input("Enter the day (e.g., Monday): ").capitalize()
    target_meal = input(
        "Enter the meal type (Breakfast/Lunch/Dinner): "
    ).capitalize()

    if target_day in meal_plan and target_meal in meals:
        new_food = input(f"Enter new food for {target_day} {target_meal}: ")
        meal_plan[target_day][target_meal] = new_food

        # Show the updated day plan
        print(f"\n Updated plan for {target_day}:")
        for meal in meals:
            print(f"  {meal}: {meal_plan[target_day][meal]}")
    else:
        print("Invalid day or meal type entered.")

print("\nHave a great, healthy week ahead!")
```

```
Welcome to food planner simulator
NOTE: Please do not input any special characters, or digits, as it will be considered as Food Not Planned

Plan for Monday:
```

```
Plan for Tuesday:
Plan for Wednesday:
Plan for Thursday:
Plan for Friday:
Plan for Saturday:
Plan for Sunday:
YOUR WEEKLY MEAL PLAN:-

📌 MONDAY
--------------------
  Breakfast: Poha
  Lunch: Rajma and chawal
  Dinner: Palak Paneer

📌 TUESDAY
--------------------
  Breakfast: Parathey
  Lunch: Fish Curry
  Dinner: Chana Masala

📌 WEDNESDAY
--------------------
  Breakfast: Poha
  Lunch: Aloo Gobhi and Roti
  Dinner: Egg bhurji

📌 THURSDAY
--------------------
  Breakfast: Poha
  Lunch: Paneer buter masala
  Dinner: Khichdi

📌 FRIDAY
--------------------
  Breakfast: gobhi ke parathey
  Lunch: Chicken curry and rice
  Dinner: Dosa

📌 SATURDAY
--------------------
  Breakfast: Idli
  Lunch: uttapam
  Dinner: dosa

📌 SUNDAY
--------------------
  Breakfast: idli
  Lunch: Set dosa
  Dinner: chicken
Have a great, healthy week ahead!
```